itthynk

SOFTWARE DEVELOPMENT INTERNSHIP PROGRAMME · COHORT 2026

Demo moved · now Monday 14 Sep, not Friday

Leave Request System — Week 4 Feedback

The code quality here is genuinely the best of any team — login, sessions, and logout are all built with real care. The lessons worth sitting with aren't about how the code is written; they're about process: the same bug being fixed and then reappearing, a branch nobody actually tried to start, and two people quietly building the same thing at once.

Repo · `cohort-2026/leave-request-system`

Sprint work · Authentication & Authorization, Mon 7 – Fri 11 Sep 2026

Demo · Monday 14 Sep 2026 (moved from Friday)

WHAT THIS DOCUMENT IS — AND ISN'T

This is **feedback, not a fix-it list.** Every item below is here to help you see where your own reasoning went wrong and think through a better way to approach it — not to hand you code to paste in. Each one points at the real file and line in your repo so you can go look at it yourselves, and some show the actual code that's there right now — not because copying a replacement is the point, but because seeing your own code named directly is more useful than a description of it. The short examples further down are illustrative, to make an idea concrete, not to be copied.

What to demo on Monday

The brief's own sequence, walked through against what this repo actually does today — from the same audit this feedback is drawn from.

Before you present: Merge PR #60 (schema) and PR #61 (seeds) first — both are clean and ready. Then fix PR #63's CORS line and missing routes, and PR #59's `ping.js` import, before merging either — merging them as committed will still break things. See Solutions 1, 2 and 5.

Step 1 Register a new account

PR #63 has conflicts

ON SCREEN

Not possible on main yet — registration still isn't merged.

IN THE CODE / TERMINAL

PR #63 has a genuinely solid `/api/auth/register` (handles the 8000-char/empty/emoji/duplicate cases correctly) but still breaks CORS and drops `/me`/`/logout`, and now conflicts with main too.

Fix it first — see Solution 1. Or: merge PR #61's seed users instead and skip live registration in the demo entirely.

Step 2 Log in

Ready

ON SCREEN

Works cleanly once you have an account — this is the strongest part of the whole system.

IN THE CODE / TERMINAL

Network tab: the login response carries `Set-Cookie: lrs_session=...; HttpOnly` — show this off, it's done exactly right.

You need a user to log in as first — see the note above about registration, or insert one manually before presenting.

Step 3 Refresh the page and confirm you're still signed in

Needs a branch merge

ON SCREEN

Stays signed in, once `feature/auth-frontend` is checked out.

IN THE CODE / TERMINAL

`GET /api/auth/me` returns the current user from the cookie.

Step 4 Visit a protected page as a guest

Needs a branch merge

ON SCREEN

Redirected to login — the `SessionGate`/`RequireActive` guards in `feature/auth-frontend` handle this correctly.

IN THE CODE / TERMINAL

None of this exists on bare main — `feature/auth-frontend` has no PR yet.

Step 5 Try an admin-only route as a normal user

Server won't boot

ON SCREEN

Should be blocked with a clear message.

IN THE CODE / TERMINAL

`requireAuth`/`requireRole` in `feature/auth-middleware` are well built and correctly split 401 vs

403 — but the branch won't start until the `ping.js` import is fixed.

See Solution 2 before relying on this branch for anything.

Step 6 **Run frontend and backend as two separate dev servers, the way you actually would** — Not fixed yet

ON SCREEN

Login will 404 — the exact cross-port issue the brief warns about.

IN THE CODE / TERMINAL

No Vite proxy, no `VITE_API_URL` anywhere in the repo (issue #46, zero commits).

See Solution 3 — this blocks testing everything else end-to-end, not only the demo.

Step 7 **Log out** — Ready

ON SCREEN

Returns to login, session ends.

IN THE CODE / TERMINAL

Logout response carries `Set-Cookie` with `Max-Age=0` — cleared server-side, not just a client redirect.

RECOMMENDED ORDER TO WORK THROUGH THIS

WHEN	ISSUE	WHERE
1st	Fix the ping.js import, and the CORS + missing routes on the registration PR	Lessons 2, 1 & 5
2nd	Pick one schema file and delete the other	Lesson 3
3rd	Add the Vite proxy so cross-port requests actually reach the backend	Lesson 4
anytime	Make the cookie's secure flag conditional before any real deployment	Lesson 6

Both open branches are currently broken as committed — fixing those two is what unblocks everything else, including being able to test the proxy fix at all.

Where this went wrong, and how to think about it differently

In the order it'll matter most — not necessarily the order you'll want to tackle it.

1 A bug that was found, and then came back

`backend/index.ts on PR #63 (feature/auth-registration, re-opened overnight)`

WHAT WENT WRONG

> The registration branch was rebuilt overnight with genuinely better validation than before — but it still has the exact same CORS setting missing that was flagged the first time around. The fix didn't carry over when the branch was redone.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
app.use(cors({ origin: "http://localhost:5173" }));
// main has: app.use(cors({ origin: "http://localhost:5173", credentials: true
}));
```

WHY IT MATTERS

> *It's like fixing a leak, then redoing the plumbing from scratch without checking the old fix was included in the new pipe layout — and the same spot starts dripping again, not because anyone did anything wrong this time, but because the earlier fix quietly got left out of the rebuild.*

A BETTER WAY TO THINK ABOUT IT

> When a branch gets significantly reworked or rebuilt, treat the earlier feedback as a checklist to re-verify against the new version — not as something that's automatically still true. "We fixed this before" is a reason to check it's still fixed, not a reason to assume it is.

2 A program that was never actually started before it was submitted

`backend/index.ts on PR #59 (feature/auth-middleware)`

WHAT WENT WRONG

> One branch has a one-line mistake — it imports a file under a name that doesn't exist. That means the server crashes immediately on startup. Every other check in that branch (role permissions, the 401/403 split) may well be built correctly, but none of it has ever actually run, because the program never gets past the first line.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
import { pingRouter } from "./routes/ping.js";
// the file that actually exists in the repo is: backend/routes/routes-ping.ts
```

WHY IT MATTERS

> *It's like writing a whole exam and turning it in without ever having tried the first question on scratch paper — the rest of the answers might be right, but nobody, including you, actually knows yet, because it was never test-run.*

A BETTER WAY TO THINK ABOUT IT

> Before opening a pull request, actually run the thing — `npm run dev`, or whatever starts the server — and confirm it boots and responds to at least one real request. "It compiles" and "it works" are different claims, and only one of them is checked by reading the code.

3 Two people quietly building the same table at once

`backend/001_users.sql vs backend/sql/001_users.sql, both on PR #60`

WHAT WENT WRONG

> The same pull request contains two different, conflicting designs for the same database table — clearly written by two different people, neither aware the other was doing the same task at the same time. Neither version is wrong on its own; the problem is that both exist together.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
-- backend/001_users.sql
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email TEXT NOT NULL UNIQUE, ...
);

-- backend/sql/001_users.sql
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT NOT NULL, ...
    CONSTRAINT uq_users_email UNIQUE (email), ...
);
```

WHY IT MATTERS

> *It's like two people separately repainting the same wall in two different colours, each thinking they were the one assigned to it. Both did the job. Now someone has to decide which coat of paint the wall actually is.*

A BETTER WAY TO THINK ABOUT IT

> Before starting a task, a quick message — "I'm taking the users table" — costs ten seconds and saves an hour of reconciling two designs later. When work does end up duplicated anyway, that's a sign to talk about it out loud, not to just pick one file and quietly delete the other.

4 A road with no bridge between the two halves of the app

`frontend/vite.config.ts · issue #46`

WHAT WENT WRONG

> The frontend and backend run on two different addresses during development, and nothing in the project tells the frontend how to reach the backend across that gap. A request that should reach the server instead gets sent to the frontend's own address and fails — which looks exactly like "login is broken," even though every individual piece of the login code is correct.

WHAT'S ACTUALLY THERE RIGHT NOW

No `server.proxy` block and no `VITE_API_URL` exist anywhere in the repo — confirmed by search. The branch created specifically for this (`3.4-make-the-cookie-travel-between-ports`) has zero commits beyond main.

WHY IT MATTERS

> *It's like two buildings on opposite sides of a river, each fully built and staffed — but nobody built the bridge, so nothing sent from one side actually reaches the other. From either building, it looks like the other side just doesn't respond.*

A BETTER WAY TO THINK ABOUT IT

> This one isn't a backend problem or a frontend problem — it's the connection between them, and it needs its own explicit piece of configuration (a dev-server proxy) rather than being something either side can fix alone. It's worth setting up early, before it gets blamed on whichever half happens to get tested first.

EXAMPLE

illustrative only

```
// frontend/vite.config.ts
server: {
  proxy: { "/api": { target: "http://localhost:4000", changeOrigin: true } },
}
// now a relative fetch("/api/...") actually reaches the backend during
development
```

5

The rebuild fixed the register logic but silently dropped two routes that already exist elsewhere

`backend/routes/auth.ts on PR #63 — only login and register are defined`

WHAT WENT WRONG

> The re-submitted registration branch's auth.ts file only defines login and register. The /me and /logout endpoints that are already live on main aren't in this file at all — so merging this branch as committed would remove two working endpoints, not just add a new one.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
export const authRouter = Router();

authRouter.post("/login", async (req, res) => { ... });
authRouter.post("/register", async (req, res) => { ... });
// that's the entire file — no authRouter.get("/me", ...), no
authRouter.post("/logout", ...)
// both exist today on main's version of this same file
```

WHY IT MATTERS

> *It's like handing in a revised version of a document that accidentally drops two paragraphs nobody meant to delete — the new part is genuinely better, but the person reading it loses information the old version had.*

A BETTER WAY TO THINK ABOUT IT

> When rebuilding a file that other work already depends on, it's worth diffing against the current main version first, to see everything that's there now — not just adding the new piece in isolation and assuming nothing else was in that file.

6 A setting that's correct today will quietly become wrong later

`backend/session.ts`

WHAT WENT WRONG

> The session cookie is currently set with secure:false, which is the right call for local development over plain HTTP. But it's not written to change automatically — if this code runs unmodified once the app is deployed over HTTPS, the cookie would still be marked insecure.

WHAT'S ACTUALLY THERE RIGHT NOW

`secure: false` is hardcoded rather than conditional — correct for local HTTP development, but would need to change before any HTTPS deployment.

WHY IT MATTERS

> *It's like leaving a note that says "unlocked for now, we're renovating" — completely reasonable during the renovation, and easy to forget to take down once the renovation is actually finished.*

A BETTER WAY TO THINK ABOUT IT

> Anything that's "correct for now, but not forever" is worth writing so the code itself remembers the exception — tying it to an environment check rather than a fixed value — so nobody has to remember to come back and change it by hand.

EXAMPLE

illustrative only

```
// currently, effectively:
secure: false,

// tie it to the environment instead, so it's automatically correct in both
places:
secure: process.env.NODE_ENV === "production",
```

Checklist — work through this before Monday

One box per lesson above, phrased as something you can actually check off. Tick it once you can point to where it's true in your own code — not once you've read the explanation. Items marked **Critical** are where the audit found a real gap; be ready to show it live, not describe it.

- [ ] **Does the re-submitted registration PR now include credentials:true in CORS?** CRITICAL
- [ ] **Does npm run dev actually boot on the middleware branch, with no import errors?** CRITICAL
- [ ] **Has one schema file been deleted, leaving a single agreed-on users table design?** CRITICAL

- [ ] **Can the frontend dev server actually reach the backend when they run on two different ports?** CRITICAL
- [ ] **Does the registration branch's auth.ts still contain /me and /logout, or did rebuilding it lose them?** CRITICAL
- [ ] Is the cookie's secure flag tied to NODE_ENV, rather than a fixed value that'll be wrong in production?